

SHIELDJOB AI: MULTI-MODAL FAKE JOB DETECTION SYSTEM

Complete Slide-by-Slide Script • Live Project Demo • Comprehensive Review Q&A • Consistency Check • Final Cheat Sheet

CANDIDATE NAME	PIN / REGD. NO.	DEPARTMENT & COLLEGE	INTERNSHIP COMPANY
Tanuku Ram Sai	24A95A0503	Dept. of CSE, Aditya University	VedXlence Innovations Pvt. Ltd., Hyderabad

PART 1 — SLIDE-BY-SLIDE SPEAKING SCRIPT (15 SLIDES)

Use this exact script while presenting the PPT. Each slide is structured for 30–90 seconds of clear, natural, spoken delivery with precise transitions and visual callout directions for diagrams and screenshots.

Slide 1 — Title Slide (Internship Project Report on ShieldJob AI)

🕒 ~30 Seconds

"Respected review panel members and faculty, a very pleasant morning/afternoon to everyone. My name is **Tanuku Ram Sai**, bearing Roll Number **24A95A0503** from the Department of Computer Science and Engineering at Aditya University. Today, I am proud to present my AICTE-certified Summer Internship project titled '**ShieldJob AI: A Multi-Modal Fake Job Detection System**'. This project was developed during my two-month internship at **VedXlence Innovations Pvt. Ltd., Hyderabad**, under the domain of Machine Learning with Python. In this presentation, I will walk you through the real-world problem of recruitment fraud, our four-layer AI/ML architecture, live deployment on the cloud, and practical browser extension integration."

Transition to Slide 2: "Let me begin by giving you a high-level executive summary of the project."

Slide 2 — Abstract

🕒 ~45-60 Seconds

"Looking at the abstract, the rapid migration of hiring processes to digital platforms like LinkedIn, Indeed, and Naukri has unfortunately resulted in an alarming increase in online employment scams. Fraudulent actors create convincing job posts with spoofed company domains and unrealistic salary offers to extort money and steal identities. To address this, **ShieldJob AI** introduces a multi-modal verification system that evaluates any job posting across **four specialized safety layers**: a Balanced Random Forest ML model, Google Gemini AI contextual auditing, automated company website scraping, and live social footprint verification. The system delivers an instant 0 to 100 risk score and safety badges through both a responsive Flask web application and a 1-click Chrome Extension. It achieves an impressive **98.05% accuracy** on 17,880 benchmark job postings."

Transition to Slide 3: "Now, let us examine the real-world background and motivation that drove this project."

Slide 3 — Introduction

 ~45-60 Seconds

"In today's digital hiring landscape, finding a job online is faster than ever, but it has also created massive vulnerabilities. Scammers now use sophisticated social engineering: they publish fake listings, conduct informal interviews over Telegram or WhatsApp, and demand upfront fees under the guise of 'training fees', 'onboarding laptop security', or 'registration charges'. Traditional keyword filters fail because modern scam job descriptions use well-written corporate language. Meanwhile, job seekers lack the technical capability to manually verify domain registration ages, SSL certificates, or official corporate filings. ShieldJob AI automates this entire cross-verification in seconds, giving students and job seekers an instant in-browser safety shield before they share sensitive personal data or hard-earned money."

Transition to Slide 4: "Let us now clearly define the exact problem statement we are solving."

Slide 4 — Problem Statement

 ~45-60 Seconds

*"Our problem statement centers around four critical industry vulnerabilities. First, thousands of fresh graduates fall victim annually to predatory recruitment scams. Second, financial extortion occurs through upfront fee demands disguised as document processing or onboarding deposits. Third, bad actors practice company impersonation through typosquatting—creating look-alike website domains that deceive unwary applicants. And fourth, existing solutions rely on single-layer checks. If you only check text, you miss fake company websites; if you only check website domains, you miss fraudulent text descriptions. Therefore, there is an urgent need for an instant, multi-modal defense tool that delivers clear **Safe**, **Suspicious**, or **Fake** verdicts in real time directly inside the user's browser."*

Transition to Slide 5: "To solve these challenges, here is an overview of the ShieldJob AI ecosystem."

Slide 5 — Project Overview

 ~60 Seconds

*"ShieldJob AI is designed as an end-to-end intelligent detection ecosystem. It is powered by **four specialized analytical pillars**: First, our **Machine Learning Pattern Matcher** weighted at 30%, trained on historical scam syntax. Second, our **Generative AI Content Inspector** using Google Gemini at 30%, which audits tone anomalies and fee-extortion semantics. Third, the **Website Authenticity Verifier** at 20%, checking live domain existence and business legitimacy. And fourth, the **Company Reputation Analyzer** at 20%, auditing LinkedIn and social footprints. These four layers combine into a unified 0 to 100 Safety Score. We provide dual interfaces: a full-featured Flask Web App for comprehensive audits and a 1-click Chrome Extension for frictionless in-browser verification with transparent explainability bullets."*

Transition to Slide 6: "Let us dive into the technical methodology and data pipeline behind these four layers."

Slide 6 — Methodology

🕒 ~60-75 Seconds

*"Our methodology follows a rigorous 5-step engineering pipeline. In **Step 1**, we preprocessed 17,880 job records from the benchmark EMSCAD dataset, extracting 12,000 unigram and bigram features via sublinear TF-IDF. In **Step 2**, we trained a Balanced Random Forest Classifier with balanced class weights to resolve the severe 4.84% scam class imbalance. In **Step 3**, we engineered structured prompts for Google Gemini 2.5 Flash to spot semantic red flags like unrealistic salaries and hidden fee requests. In **Step 4**, we integrated BeautifulSoup for automated website scraping and Brave Search/Google CSE for live social verification. Finally, in **Step 5**, our aggregation engine calculates the final Safety Score as 30% ML plus 30% Gemini AI plus 20% Website plus 20% Social verification."*

Transition to Slide 7: "Now, let us examine the complete system architecture diagram."

Slide 7 — System Architecture

🕒 ~75-90 Seconds

*"This slide illustrates the multi-pillar architecture of ShieldJob AI. When the user inputs a job description and optional company URL, our system distributes the payload into four parallel processing channels. The **ML model** analyzes statistical n-gram patterns. Concurrently, **Gemini AI** evaluates the contextual legitimacy and recruiter tone. Simultaneously, the **Website Check** scrapes the domain with BeautifulSoup to verify active operations, while the **Social Check** searches LinkedIn and public scam databases. All four independent probability outputs feed into our **Score Aggregation Engine**, producing a final weighted score mapped to three clear badges: Emerald for Legitimate (70 to 100), Amber for Suspicious (50 to 69), and Rose/Red for Likely Fake (below 50)."*

What to point at on Slide 7:

- **Top Block:** Point to "User Input" where job description and company URL enter.
- **Middle 4 Boxes:** Point to the 4 parallel pillars (30% ML, 30% Gemini AI, 20% Website, 20% Social).
- **Bottom Aggregation Engine:** Point to the weighted formula and the 3 final classification badge outputs.

Slide 8 — System Structure & Workflow

🕒 ~60-75 Seconds

"Here is the step-by-step user workflow in practice. First, while browsing portals like LinkedIn, Naukri, or Unstop, the user encounters a job listing. Second, they open the 1-click ShieldJob Chrome Extension and paste the description. Third, the extension dispatches a secure HTTPS payload to our Flask REST API hosted on Render. Crucially, zero API keys are exposed in the client extension. Fourth, the cloud backend triggers the four-layer multi-modal audit in parallel. And fifth, within two to three seconds, the user receives an instant diagnostic verdict with the Safety Score gauge, risk breakdown, and actionable red flag warnings."

What to point at on Slide 8: Trace the 5 numbered steps from left to right: (1) Browse Job Portal → (2) Chrome Extension Popup → (3) Flask Cloud API on Render → (4) Parallel Multi-Layer Scan → (5) Instant Verdict & Red Flags.

Slide 9 — Implementation & Tech Stack

🕒 ~60 Seconds

"Looking at our implementation technologies: On the backend, we utilized **Python 3.11 with Flask REST API**, Gunicorn WSGI server, and CORS handling. For Machine Learning, we used **Scikit-Learn**, implementing TF-IDF vectorization and Random Forest classification with Joblib serialization. For Generative AI, we integrated the **Google Gemini API** using the official Python SDK. For live web and reputation verification, we used **BeautifulSoup** and search APIs. On the frontend, we built a modern glassmorphism UI with HTML5, Vanilla CSS3, ES6 JavaScript, and developed the **Chrome Extension under Manifest V3** standards. The complete system is deployed live on **Render Cloud** with automated CI/CD."

Transition to Slide 10: "Let us now examine the benchmark experimental results and validation metrics."

Slide 10 — Results: Model Accuracy & Proof

🕒 ~75-90 Seconds

"We evaluated our machine learning pipeline using **5-Fold Stratified Out-of-Fold Cross-Validation** across all 17,880 dataset samples to guarantee zero data leakage. The results show outstanding performance: an overall **Accuracy of 98.05%**, a **Scam Recall (Catch Rate) of 81.29%**, a **Precision of 79.10%**, an **F1-Score of 80.18%**, and a **ROC-AUC of 98.49%**. Looking at our confusion matrix, out of 17,014 genuine jobs, 16,828 were correctly approved with only a 1.1% false alarm rate. Out of 866 real fraudulent postings, our model successfully detected 704 scams. This demonstrates that our calibrated threshold of 0.24 provides a robust balance between high scam detection and minimal false alarms."

What to point at on Slide 10:

- **Top Metrics:** Point to 98.05% Accuracy and 81.29% Recall.
- **Confusion Matrix:** Highlight True Negatives (16,828 real jobs approved) and True Positives (704 scams caught out of 866). Emphasize the low False Positive rate (186 / 1.1%).

Slide 11 — Results: Live App & Extension in Action

🕒 ~60-75 Seconds

"This slide demonstrates our working software artifacts in action. On the left, you can see the **Google Chrome Extension** verifying a live job posting from Unstop for a Cisco position. The extension instantly displays a verified Legitimate status with a 75.8 Safety Score and 91% ML confidence. On the right, you can see our **Scan History Dashboard** on the web application, which logs analyzed jobs with interactive safety badges, timestamped audit summaries, and one-click re-scan capabilities. This proves that ShieldJob AI is not just a theoretical model, but a fully functional, cloud-deployed product."

What to point at on Slide 11:

- **Left Screenshot:** Point to the Chrome Extension popup showing the 75.8 score and green Legitimate badge.
- **Right Screenshot:** Point to the web dashboard scan history table showing recent job scans and color badges.

Slide 12 — Challenges & Learning Outcomes

 ~60 Seconds

*"During development, we solved three major technical challenges. First, **extreme class imbalance**—scams represent only 4.84% of the dataset. We resolved this using balanced class weighting and stratified sampling. Second, **unstructured text data** with missing fields, which we standardized through regex pipelines and sublinear TF-IDF scaling. Third, **cloud cold starts and external API latency**, which we addressed by executing API calls in parallel and implementing graceful fallback modes so the system never crashes. Through this project, I gained hands-on industry mastery in end-to-end ML pipelines, LLM prompt engineering, Manifest V3 browser extension development, and production cloud deployment."*

Transition to Slide 13: "Let us conclude with the project summary and future scope."

Slide 13 — Conclusion & Future Scope

 ~60 Seconds

*"In conclusion, ShieldJob AI proves that combining classical Machine Learning with Generative AI and live digital footprint auditing creates a robust, multi-layered defense against modern recruitment fraud. By delivering both a web platform and a Chrome Extension, we make enterprise-grade verification accessible to every student with a single click. In the future, we plan to implement **automated DOM scraping** to highlight scam postings on job portals automatically without copy-pasting, expand to **multilingual NLP models** for regional Indian languages, integrate official corporate registry checks via the Ministry of Corporate Affairs (MCA), and develop a WhatsApp bot for instant mobile verification."*

Transition to Slide 14 & 15: "Here are our key academic and technical references."

Slide 14 & 15 — References & Thank You

 ~30 Seconds

"Our work is grounded in established research, including Scikit-Learn literature by Pedregosa et al., Google's Gemini technical papers, Chrome Manifest V3 documentation, and the benchmark EMSCAD dataset. With this, I conclude my presentation. I want to express my sincere gratitude to my project guide, Mr. M. Srinu, Head of the Department Dr. T. Sudha Rani, faculty members, and VedXlence Innovations for their support. I am now open to your questions and valuable feedback. Thank you!"

PART 2 — LIVE PROJECT DEMONSTRATION SCRIPT (12 SECTIONS)

Follow this step-by-step spoken script when demonstrating the live ShieldJob AI web application and Chrome extension to the reviewers.

1. Demo Introduction

"Now, I would like to demonstrate the live, fully functional ShieldJob AI system. The web application is deployed on the Render cloud platform at <https://fake-job-detection-iuxn.onrender.com>, and we also have our Chrome Extension running locally in developer mode under Manifest V3 standards. I will show how our four-pillar system processes both genuine and fraudulent job postings in real time."

2. Application / Home Page Interface

"Here is the main landing page and analyzer dashboard of ShieldJob AI. It features a modern, responsive cyber-themed glassmorphism interface built with Vanilla CSS. At the top, we have our navigation bar with quick links to the Analyzer, System Architecture, Live Proof & Accuracy metrics, and Scan History. The primary input card provides fields for the Job Title, Job Description, and an optional Company Website URL for live domain auditing."

3. User Input

"To demonstrate, let us take a suspicious job posting that promises a high salary for a data entry role but mentions an upfront 'laptop training kit fee' and provides an unverified contact email. I paste the job title 'Remote Data Entry Associate' and the full job description into the text area, and enter the provided company website URL. Now, I will click the 'Analyze Job' button."

4. Analysis Process

"When I click 'Analyze Job', the frontend sends an asynchronous POST request to our Flask backend route. The backend immediately dispatches four concurrent analysis tasks: tokenizing the text for our Random Forest model, querying the Google Gemini 2.5 Flash API with custom fraud-auditing prompts, initiating BeautifulSoup to scrape and inspect the company domain, and querying search APIs for social footprints."

5. Machine Learning Prediction Layer

"First, our serialized TF-IDF vectorizer converts the input text into a 12,000-dimensional unigram/bigram feature vector with sublinear term-frequency scaling. Our Balanced Random Forest model evaluates statistical n-gram probabilities against the 17,880 historical patterns learned during training, outputting a machine learning safety probability score."

6. Gemini AI Contextual Validation Layer

"Simultaneously, Google Gemini 2.5 Flash acts as a semantic fraud investigator. While the ML model looks at word frequencies, Gemini reads the text contextually. In this sample, Gemini immediately flags the requirement of an upfront training deposit, the inflated salary of \$45/hour for basic typing, and informal communication channels, returning a detailed JSON risk payload."

7. Website Analysis Layer

"Next, our Website Analyzer service fetches the target URL using BeautifulSoup with custom browser headers. It evaluates domain structure, meta-tags, and body content to verify whether the domain belongs to an active, legitimate corporate entity or is an empty shell page registered to deceive applicants."

8. Social Media Analysis Layer

"Concurrently, the Social Media Verifier queries Brave Search and Google CSE for the company's verified LinkedIn page, Glassdoor employee reviews, and scam alert mentions on forums like Reddit. If search quotas are limited, it automatically falls back to Gemini AI Knowledge Mode to audit public reputation."

9. Final Risk Score & Verdict Presentation

"Our Score Aggregation Engine combines all four weighted outputs: 30% ML, 30% Gemini AI, 20% Website, and 20% Social. As you can see on the screen, the system calculated an overall Safety Score of **28 out of 100**, displaying a prominent **Rose/Red 'Likely Fake'** alert badge with an interactive circular SVG gauge."

10. Explanation / Red Flags Diagnostic

"Below the safety score gauge, ShieldJob AI provides transparent, human-readable explainability bullets. It explicitly lists the exact red flags detected: 'Demands upfront onboarding fee', 'Unrealistic salary for entry-level work', 'Domain lack of corporate history', and 'High statistical similarity to known scam templates'. This transparency helps the student understand exactly why the job was flagged."

11. Other Important Features (Scan History & Chrome Extension Demo)

"If we scroll down, you can see the **Scan History Dashboard**, which logs every analyzed job with timestamps, score badges, and re-scan options. Now, let me switch to a live job portal in Google Chrome. When I click the **ShieldJob AI Chrome Extension** icon, a compact popup opens. I paste the job text, click Analyze, and within 2 seconds, the extension renders the identical 4-pillar risk score directly inside the browser window via our lightweight REST API."

12. Demo Conclusion

"This completes the live demonstration of ShieldJob AI. We have seen how the Flask web application and the Manifest V3 Chrome Extension seamlessly leverage Machine Learning, Gemini AI, website scraping, and social auditing to protect job seekers in real time. Thank you, and I am ready for the technical Q&A session."

PART 3 — COMPREHENSIVE INTERNSHIP REVIEW Q&A (45 TOPIC AREAS)

Every question contains a concise, technically correct, natural answer tailored for a 2-5 sentence spoken response during review.

01 General Project: What is ShieldJob AI in simple terms?

"ShieldJob AI is a multi-modal cybersecurity and fake job detection platform. It uses a four-pillar approach—combining classical Machine Learning, Google Gemini Generative AI, live website scraping, and social reputation auditing—to determine whether an online job posting is genuine, suspicious, or a scam. It delivers instant 0 to 100 safety scores through a web app and Chrome extension."

02 Internship & Organization: Where did you do your internship and for how long?

"I completed a 2-month AICTE-certified internship from May 6, 2026, to July 5, 2026, at VedXlence Innovations Pvt. Ltd., located in Hyderabad. The internship domain was Machine Learning with Python, structured into an initial technical upskilling phase followed by full-stack real-world project implementation."

03 Problem Statement: What specific problem does your project address?

"With hiring moving online, scammers create convincing job postings on platforms like LinkedIn and Naukri to steal personal identity data and extort upfront registration or training fees. Traditional keyword filters fail to catch modern scam phrasing, and manual background checks are too difficult for everyday students. ShieldJob AI provides an automated, real-time safety verification tool."

04 Motivation: What inspired you to choose fake job detection?

"As college students entering the placement season, my peers and I frequently encounter unsolicited recruitment offers demanding registration deposits or suspicious Telegram interviews. Seeing fellow graduates lose money and compromise their personal credentials motivated me to build a practical, accessible AI defense tool."

05 Objectives: What were the primary technical objectives of your project?

"Our primary objectives were to train a high-accuracy machine learning model on 17,880 historical job records, integrate Google Gemini AI for contextual fraud auditing, implement live website and social verification, and deploy the solution as both a responsive Flask web app and a 1-click Chrome Extension."

06 Proposed Solution: How does your solution differ from existing spam filters?

"Existing solutions rely on single-layer text filters or static keyword blacklists that miss sophisticated fraud. ShieldJob AI uses a multi-modal 4-pillar approach that simultaneously checks statistical text syntax, LLM semantic reasoning, live domain scraping, and social media footprint, making it virtually impossible for scammers to bypass all safety layers."

07 Technologies Used: What is the complete technology stack of your project?

"On the backend, we use Python 3.11 with Flask, Gunicorn WSGI, Scikit-Learn for ML, Google Gemini 2.5 Flash API for GenAI, and BeautifulSoup for web scraping. On the frontend, we use HTML5, Vanilla CSS3 with glassmorphism, JavaScript ES6, and Chrome Extension Manifest V3, all hosted on Render Cloud."

08 Technology Selection: Why did you choose Flask instead of Django or FastAPI?

"Flask is lightweight, modular, and offers minimal overhead for serving our Single-Page Application and microservice REST API endpoints. Since our application does not require a heavy relational ORM or built-in authentication tables, Flask allowed us to build high-speed asynchronous prediction routes with clean architectural boundaries."

09 System Architecture: Explain the 4 pillars of your architecture.

"The architecture consists of four weighted analytical pillars: Layer 1 is the Balanced Random Forest ML model (30%), Layer 2 is the Gemini 2.5 Flash semantic auditor (30%), Layer 3 is the BeautifulSoup website scraper (20%), and Layer 4 is the search-engine social footprint auditor (20%). Their outputs are aggregated into a final 0 to 100 Safety Score."

10 Project Workflow: What happens internally from the moment a user submits a job?

"The Flask backend receives the payload and triggers four parallel tasks. The text is vectorized through TF-IDF and passed to Random Forest, Gemini evaluates semantic red flags via API, BeautifulSoup scrapes the company website, and search APIs query LinkedIn. The Score Aggregation Engine computes the weighted score and returns a JSON verdict with explainability bullets."

11 Machine Learning: What role does classical ML play in your system?

"Machine learning provides high-speed statistical pattern matching. It analyzes term frequency distributions across 17,880 historical job descriptions to instantly calculate the statistical probability that a text matches known recruitment fraud phrasing without incurring API costs."

12 NLP: What Natural Language Processing techniques did you apply?

"We implemented text normalization including lowercase conversion, URL tokenization, special character cleaning, and stopwords filtering. We then applied TF-IDF feature extraction with unigram and bigram tokenization and sublinear term-frequency scaling across a vocabulary of 12,000 features."

13 TF-IDF: Why did you use TF-IDF and what specific hyperparameters did you choose?

"TF-IDF converts unstructured text into numerical vectors by weighing terms based on frequency in a document versus frequency across all documents. We configured `max_features=12,000`, `ngram_range=(1,2)` to capture two-word phrases like 'wire transfer', `sublinear_tf=True` to dampen high-frequency terms, and `min_df=2` to eliminate noise."

14 ML Models: Which machine learning algorithm did you use and why?

"We used the **Random Forest Classifier** with 100 decision trees. Random Forest is an ensemble learning method that combines multiple decorrelated decision trees, preventing overfitting on high-dimensional text data and providing robust generalization."

15 Model Selection: Why did you select Random Forest over Logistic Regression or Naive Bayes?

"While Naive Bayes assumes strict feature independence and Logistic Regression finds linear decision boundaries, Random Forest captures complex non-linear feature interactions between multiple job attributes and provides built-in support for balanced class weighting."

16 Training & Testing: How did you train and validate your machine learning model?

"We trained the model on 17,880 postings from the EMSCAD dataset using **5-Fold Stratified Out-of-Fold Cross-Validation**. Stratified splitting ensured each fold preserved the exact 4.84% scam ratio, and out-of-fold predictions guaranteed zero data leakage between training and evaluation."

17 Accuracy & Evaluation: What are your key evaluation metrics and results?

"Across all 17,880 benchmark samples, our model achieved **98.05% Accuracy, 79.10% Precision, 81.29% Scam Recall, 80.18% F1-Score**, and **98.49% ROC-AUC**, catching 704 out of 866 real fraud cases with only 186 false alarms."

18 Gemini AI: Which Generative AI model did you integrate and how?

"We integrated **Google Gemini 2.5 Flash** via the official `google-generativeai` Python SDK. It is invoked on the backend with engineered prompts and strict JSON schema constraints to return deterministic fraud assessments and explainability bullets."

19 AI Validation: Why is Gemini AI necessary if you already have an ML model?

"Classical ML only detects statistical n-gram patterns, so if a scammer writes a grammatically perfect job posting, ML alone might miss it. Gemini AI provides deep semantic understanding—it evaluates whether the salary is realistic for the duties, identifies subtle fee-extortion phrasing, and checks recruiter professionalism."

20 Website Analysis: How does your system verify company websites?

"The Website Analyzer uses BeautifulSoup with custom browser headers to fetch the target domain's HTML. It extracts text, title, and meta tags, and prompts Gemini to audit whether the site represents an active commercial business or an empty shell domain created for typosquatting."

21 Social Media Analysis: How do you verify social media and company reputation?

"Our Social Media Verifier extracts the company name and queries Brave Search and Google Custom Search Engine for verified LinkedIn company pages, employee counts, Glassdoor reviews, and Reddit scam reports. If search quotas are unavailable, it seamlessly switches to Gemini AI Knowledge Mode."

22 Risk Scoring: What is your exact mathematical formula for the Safety Score?

"The formula is: $\text{Safety Score} = (\text{ML_Score} * 0.30) + (\text{Gemini_Score} * 0.30) + (\text{Website_Score} * 0.20) + (\text{Social_Score} * 0.20)$. Scores from 70 to 100 are Legitimate (Safe), 50 to 69 are Suspicious (Proceed with Caution), and 0 to 49 are Likely Fake (High Risk)."

23 Flask / Backend: How is the Flask application structured?

"The Flask app is modularized with clean separation: `app.py` handles routing and CORS, `services/` contains dedicated modules for ML inference, Gemini API, website scraping, and social auditing, and `models/` stores serialized pickle artifacts and evaluation metrics."

24 Frontend: How did you design the user interface?

"The frontend is built using clean semantic HTML5 and custom Vanilla CSS3 with glassmorphism aesthetics, dynamic SVG gauge animations, and responsive layouts. Asynchronous JavaScript fetch handles non-blocking API communication without page reloads."

25 APIs: What external APIs are used and how do you protect API keys?

"We use the Google Gemini API and Brave Search / Google CSE APIs. All API keys are securely stored as server-side environment variables on the Render cloud platform. The Chrome Extension communicates only with our Flask API, ensuring zero keys are exposed to clients."

26 Database: How do you handle scan history and persistence?

"Scan history is stored in-memory and managed through client session and local storage with CSV export capabilities. For a production enterprise deployment, this can be seamlessly migrated to PostgreSQL with SQLAlchemy ORM."

27 Data Preprocessing: What steps were taken to clean the raw dataset?

"We concatenated title, company profile, description, requirements, and benefits into a single text block, removed missing values, lowercased all text, stripped HTML and URLs using regular expressions, and removed standard English stopwords before TF-IDF vectorization."

28 Dataset: Tell me about the dataset used for training.

"We used the benchmark Employment Scam Aegean Dataset (EMSCAD / Kaggle Fake Job Postings). It consists of 17,880 real-world job postings with 18 metadata attributes, containing 17,014 legitimate jobs (95.16%) and 866 confirmed fraudulent postings (4.84%)."

29 Feature Extraction: Why did you use unigrams and bigrams together?

"Unigrams capture isolated keywords like 'fee' or 'wire', but scammers often use innocent individual words that become suspicious only in combination, such as 'wire transfer', 'registration deposit', or 'immediate joining'. Bigrams capture these critical two-word contextual relationships."

30 Deployment: How and where is ShieldJob AI deployed?

"The backend is deployed as a cloud web service on the Render platform utilizing Python 3.11, Gunicorn WSGI server, automated environment variable injection, and CORS headers allowing secure connections from the Chrome Extension."

31 Testing: How did you test the backend and extension?

"We performed unit testing on the ML inference pipeline with sample edge cases, API contract testing on `/api/extension/analyze` using Postman and curl, and end-to-end user testing of the Chrome Extension popup across LinkedIn, Indeed, and Unstop job pages."

32 Security: What security practices did you implement?

"We enforced Manifest V3 Content Security Policy (CSP) on the extension, completely isolated API secrets on the cloud backend, sanitized user inputs to prevent XSS attacks, and implemented strict timeout protections on all external web scraping and API requests."

33 Challenges Faced: What was the single biggest technical challenge you faced?

"The biggest challenge was the extreme 95.16% to 4.84% class imbalance in the training data. A naive model would simply predict 'legitimate' 100% of the time and get 95% accuracy while catching zero scams. We solved this using balanced class weighting and threshold calibration."

34 Problems During Development: Did you face any API rate-limiting or latency issues?

"Yes, external API calls to Gemini and search engines initially caused response delays. We resolved this by optimizing parallel requests and implementing graceful fallback mechanisms so that if an external API is slow or quota-limited, the system safely computes an ML-weighted verdict."

35 Solutions to Challenges: How did you calibrate your decision threshold to 0.24?

"Through 5-fold out-of-fold validation, we plotted precision-recall curves. At the standard 0.50 threshold, scam recall was low because of class imbalance. Calibrating the threshold to 0.24 elevated our scam recall to 81.29% while keeping false alarms at just 1.1%."

36 Limitations: What are the current limitations of ShieldJob AI?

"Current limitations include requiring an active internet connection for cloud API calls, dependency on external search engine quotas for social verification, and the inability to inspect private encrypted chat groups like Telegram if scammers conduct interviews offline."

37 Future Scope: What enhancements would you build next?

"In the future, we plan to add automatic DOM scraping in the Chrome extension to highlight scam jobs on web pages without copy-pasting, support multilingual NLP for regional Indian languages, integrate official MCA corporate registry verification, and build a WhatsApp verification bot."

38 Personal Contribution: What was your specific individual contribution?

"I personally designed and developed the entire pipeline: performing exploratory data analysis, training and tuning the Balanced Random Forest model, engineering Gemini AI prompts, building the BeautifulSoup scraper, developing the Flask web app and Chrome Extension, and deploying on Render."

39 Internship Experience: How did this internship help your technical growth?

"The internship at VedXlence bridged the gap between academic theory and production software engineering. I gained hands-on experience handling imbalanced ML datasets, integrating modern Generative AI SDKs, building secure browser extensions, and managing cloud deployments."

40 Project Demo Questions: Why does your demo show both ML Confidence and Safety Score?

"ML Confidence represents the statistical probability output from the Random Forest model alone, while the Safety Score is the final comprehensive 4-pillar aggregated metric. Showing both provides full diagnostic transparency to the user."

41 Diagrams & Screenshots: Why does your architecture use equal 30% weights for ML and AI?

"We assign 30% to ML for rapid historical pattern matching and 30% to Gemini AI for deep semantic reasoning. Giving them equal weight balances statistical rigor with semantic intelligence, while the remaining 40% is split equally between live website and social footprint verification."

42 Questions About Results: Why is your 98.05% accuracy significant if the baseline is 95.16%?

"In an imbalanced dataset where 95.16% of samples are legitimate, a useless model that approves everything achieves 95.16% accuracy but has 0% recall on scams. Our model achieves 98.05% accuracy while actively catching 81.29% of all fraud cases, proving genuine predictive power."

43 Challenging Technical Decisions: Why not use a Deep Learning model like LSTM or BERT?

"While BERT and LSTMs are powerful, they require significant GPU compute resources, high memory footprints, and slow inference times that cause unacceptable latency on free cloud tiers. Random Forest with TF-IDF delivers 98.05% accuracy with sub-millisecond inference and a lightweight 5.6 MB model size."

44 Difficult Reviewer Questions: What if a brand new startup has no website or LinkedIn footprint?

"Our system handles this gracefully. If a company lacks a website or LinkedIn page, the Website and Social layers return neutral baseline scores, while the ML model and Gemini AI evaluate the job text itself. If the job description is clean and realistic, it receives a Suspicious/Proceed with Caution badge rather than being falsely flagged as Fake."

45 Basic Understanding: What is the core difference between Precision and Recall in your project?

"Recall measures our scam catch rate—out of all real fake jobs, how many did we catch (81.29%). Precision measures our flagging accuracy—when we alert a user that a job is fake, how often are we correct (79.10%). Both are balanced by our F1-score of 80.18%."

PART 4 — QUESTIONS SPECIFICALLY LIKELY TO BE ASKED (RANKED)

● Very High Probability Questions (Must-Prepare)

1. What is the 4-layer architecture and what are the exact weights?

"ShieldJob AI uses four analytical layers: Machine Learning Random Forest at 30%, Google Gemini 2.5 Flash GenAI at 30%, Website Verification at 20%, and Social Media Footprint at 20%. The formula is: `Safety Score = 0.30(ML) + 0.30(AI) + 0.20(Web) + 0.20(Social)`, mapped to Safe (70-100), Suspicious (50-69), and Fake (0-49)."

2. How did you handle the class imbalance in the dataset?

"The dataset has 17,014 real jobs (95.16%) and only 866 fake jobs (4.84%). We resolved this by using a Balanced Random Forest with `class_weight='balanced'`, which automatically penalizes misclassifications of minority scam samples inversely proportional to their frequency (~19.6x weight), and calibrated our decision threshold to 0.24."

3. Why do you need both Machine Learning and Gemini AI?

"Machine learning performs fast, low-cost statistical pattern matching on n-grams. Gemini AI provides deep semantic understanding to catch subtle fraud that syntax alone misses—such as disguised registration fee demands or unrealistic salary-to-skill ratios."

4. What dataset did you use and what are its key statistics?

"We used the benchmark EMSCAD dataset from Kaggle containing 17,880 total job postings across 18 features. It has 17,014 legitimate samples (95.16%) and 866 fraudulent samples (4.84%)."

5. What are your model's accuracy, precision, and recall numbers?

"Evaluated via 5-fold stratified cross-validation, the model achieved 98.05% Accuracy, 79.10% Precision, 81.29% Recall (catching 704 out of 866 scams), an 80.18% F1-score, and a 98.49% ROC-AUC."

6. How does the Chrome Extension communicate with your system?

"The Chrome Extension is built on Manifest V3. When a user clicks 'Analyze', the extension sends an HTTPS POST request to our Flask REST endpoint `/api/extension/analyze` on Render, which processes the multi-layer scan and returns JSON results in under 2 seconds."

● High Probability Questions (Commonly Asked)

7. Why did you choose Random Forest instead of Deep Learning?

"Random Forest with TF-IDF provides outstanding accuracy (98.05%) with minimal memory usage (5.6 MB model size) and sub-millisecond inference time. Deep learning models like BERT require heavy GPU infrastructure that is impractical for free cloud tiers and real-time browser extension responses."

8. What features did you extract from the job postings?

"We concatenated title, company profile, description, requirements, and benefits into a single text corpus and extracted 12,000 unigram and bigram TF-IDF features with sublinear term-frequency scaling."

9. How do you scrape and verify company websites without getting blocked?

"We use BeautifulSoup with custom browser User-Agent headers, timeout limits, and domain parsing to extract visible page text, and then pass the content to Gemini AI to assess if it represents active business operations."

10. What happens if the Gemini API or search quota is exhausted?

"Our backend implements graceful fallback handlers. If Gemini or search APIs fail, the system automatically recalibrates the safety score using our local Random Forest ML model, ensuring zero server crashes and continuous uptime."

11. What is the role of the decision threshold 0.24?

"Due to class imbalance, the standard 0.50 threshold misses too many scams. By setting the threshold to 0.24 through out-of-fold validation, our scam catch rate rose to 81.29% while keeping false alarms to a low 1.1%."

12. What was your internship organization and duration?

"I interned at VedXlence Innovations Pvt. Ltd., Hyderabad, for 2 months from May 6, 2026, to July 5, 2026, in the Machine Learning with Python domain."

● Possible Questions (Reviewer Dependent)

13. Which UN Sustainable Development Goals (SDGs) does your project map to?

"ShieldJob AI maps to SDG 8 (Decent Work and Economic Growth - Target 8.8: protecting labour rights and safe working environments), SDG 9 (Industry, Innovation & Infrastructure), and SDG 16 (Peace, Justice and Strong Institutions - Target 16.4: combating illicit fraud)."

14. What is sublinear term-frequency scaling in TF-IDF?

"Sublinear scaling replaces raw term frequency tf with $1 + \log(tf)$. This prevents terms that appear 20 times in a single job posting from having 20 times the weight of a term appearing once, improving classification robustness."

15. How do you prevent API key leaks in your Chrome Extension?

"The Chrome Extension contains zero API keys or business secrets. It only sends payloads to our Flask backend on Render, where all external API keys are secured as protected environment variables."

PART 5 — IMPORTANT POINTS TO MEMORIZE (CHEAT FACTS)

Parameter / Category	Exact Verified Fact (From PPT & Report)
Candidate Details	Tanuku Ram Sai PIN: 24A95A0503 Dept. of CSE, Aditya University, Surampalem
Internship Organization	VedXlence Innovations Pvt. Ltd., Hyderabad, Telangana, India (https://www.vedxlence.in/)
Internship Duration & Domain	2 Months (May 6, 2026 – July 5, 2026) Domain: Machine Learning with Python
Project Guide & HOD	Guide: Mr. M. Srinu, Asst. Professor HOD: Dr. T. Sudha Rani, Professor & Head, Dept. of CSE
Project Title	ShieldJob AI: Multi-Modal Fake Job Detection System
Dataset & Sample Count	EMSCAD / Kaggle Fake Job Postings: 17,880 Total (17,014 Legitimate / 95.16% vs 866 Fraudulent / 4.84%)
NLP / Feature Extraction	TfidfVectorizer: max_features=12,000, ngram_range=(1,2), sublinear_tf=True, min_df=2
ML Model & Hyperparameters	RandomForestClassifier(n_estimators=100, class_weight='balanced', random_state=42), Threshold = 0.24
Generative AI Model	Google Gemini 2.5 Flash via google-generativeai SDK
4-Pillar Scoring Formula	<code>Safety Score = 0.30(ML) + 0.30(AI) + 0.20(Website) + 0.20(Social)</code>
Score Categories & Badges	70–100: Legitimate (Emerald) 50–69: Suspicious (Amber) 0–49: Likely Fake (Rose/Red)
Verified Accuracy & Metrics	Accuracy: 98.05% Precision: 79.10% Recall: 81.29% F1-Score: 80.18% ROC-AUC: 98.49%
Confusion Matrix Values	TN: 16,828 (98.9%) FP: 186 (1.1%) FN: 162 (18.7%) TP: 704 (81.3% scams caught)
Web & Browser Tech Stack	Python 3.11, Flask REST API, Gunicorn, HTML5, Vanilla CSS3 Glassmorphism, JS ES6, Chrome MV3
Live Cloud URL & Repo	Live App: https://fake-job-detection-iuxn.onrender.com/#analyzer GitHub: Coder-sai2004/Fake_Job_Detection
UN SDG Mappings	SDG 8 (Decent Work & Economic Growth - Target 8.8), SDG 9 (Innovation), SDG 16 (Target 16.4)

PART 6 — POSSIBLE TRICKY QUESTIONS & HONEST SPOKEN ANSWERS

Q1: Why did you choose Random Forest instead of XGBoost or a Transformer model?

"While XGBoost and Transformers are powerful, Random Forest was selected because it naturally handles high-dimensional sparse text vectors, provides built-in balanced class weighting without extreme hyperparameter sensitivity, trains in seconds, and serializes to a lightweight 5.6 MB file that runs with sub-millisecond latency on cloud servers."

Q2: Why use both classical ML and Gemini AI? Isn't Gemini alone enough?

"Relying solely on Gemini AI would introduce significant API latency, recurring financial costs, and vulnerability to network rate limits. Classical ML provides a fast, zero-cost first line of statistical defense, while Gemini provides high-level semantic reasoning for nuanced fraud. Together, they create a robust, cost-effective hybrid system."

Q3: How do you know your model is not overfitting?

"We validated the model using 5-Fold Stratified Out-of-Fold Cross-Validation across all 17,880 samples, ensuring evaluation was always performed on unseen data. Additionally, our ensemble of 100 decorrelated trees and a minimum document frequency constraint (min_df=2) prevents individual noisy words from overfitting."

Q4: What happens if the Gemini AI gives an incorrect response or hallucinates?

"Gemini AI contributes only 30% of the overall score and is constrained with strict system prompts and JSON schema outputs. Even if the LLM output is imperfect, the remaining 70%—comprising our Random Forest model, website domain scraping, and social footprint verification—counterbalances the assessment."

Q5: What happens if a legitimate new startup has no company website or LinkedIn page?

"When website or social data is unavailable, those modules return neutral baseline scores. The system relies primarily on the job text evaluated by ML and Gemini. If the description has realistic compensation and no fee demands, it receives a 'Suspicious / Proceed with Caution' badge rather than being falsely classified as a fake scam."

Q6: What happens when the input text is completely different from your training data?

"If novel vocabulary appears, the TF-IDF vectorizer captures available token overlaps, but our Gemini AI layer steps in to understand the semantic context and meaning. Because Gemini is a general-purpose LLM, it evaluates the logical coherence and scam indicators regardless of unseen vocabulary."

Q7: What is the most difficult part of the project that you personally implemented?

"The most challenging aspect was architecting the multi-modal score aggregation engine and handling class imbalance. Calibrating the decision threshold to 0.24 through cross-validation and building seamless API fallback mechanisms required deep debugging to ensure high scam recall without sacrificing speed."

Q8: Why should someone trust your system over their own judgment?

"ShieldJob AI performs multi-dimensional checks that a human cannot easily do in a few seconds—such as calculating n-gram probability against 17,880 historical patterns, analyzing domain metadata, and checking cross-platform complaints simultaneously—and provides transparent explainability bullets to help the user make an informed decision."

Q9: What are the security concerns regarding your Chrome Extension?

"The Chrome Extension is built strictly under Manifest V3, adhering to modern Content Security Policies. It does not store or process user passwords or personal browsing history, and communicates only with our secured HTTPS backend endpoint on Render."

Q10: How can the project be scaled for millions of users?

"To scale, the Flask microservice can be containerized using Docker and deployed on an autoscaling Kubernetes cluster behind an Nginx load balancer, with Redis caching for frequently queried job postings and company domains."

PART 7 — PPT / REPORT CONSISTENCY CHECK TABLE

This comparison verifies all project facts between the Presentation PPT and the Internship Report to ensure you never make contradictory statements during review.

Issue / Parameter	PPT Claim	Report Claim	What You Should Speak
Internship Company	Not explicitly named on slide titles (focuses on Aditya University)	VedXlence Innovations Pvt. Ltd., Hyderabad (AICTE Certified)	Speak: "Carried out at VedXlence Innovations Pvt. Ltd., Hyderabad."
Internship Dates	Academic Year 2025–2026	May 6, 2026 – July 5, 2026 (2 Months)	Speak: "2-month internship from May 6, 2026 to July 5, 2026."
Gemini Model Version	"Google Gemini AI"	"Google Gemini 2.5 Flash"	Speak: "Google Gemini 2.5 Flash via official Python SDK."
Scoring Weights	30% ML, 30% Gemini AI, 20% Website, 20% Social	30% ML, 30% Gemini AI, 20% Website, 20% Social	Consistent: Use 30% ML, 30% AI, 20% Web, 20% Social.
Score Categories	Legitimate (Safe), Suspicious, Likely Fake	Safe (≥ 70), Suspicious (50–69), Likely Fake (< 50)	Consistent: Use 70–100 Safe, 50–69 Suspicious, 0–49 Fake.
Dataset Size & Source	17,880 benchmark job postings (EMSCAD)	17,880 postings (17,014 real / 866 fake) from EMSCAD / Kaggle	Consistent: 17,880 samples (95.16% real, 4.84% fake).
Model & Hyperparams	Random Forest + TF-IDF	Balanced Random Forest (n_estimators=100, class_weight='balanced', threshold=0.24, max_features=12,000)	Speak: Balanced Random Forest with 100 trees, 12K TF-IDF features.
Evaluation Metrics	Accuracy: 98.05%, Recall: 81.29%, Precision: 79.10%, ROC-AUC: 98.49%	Accuracy: 98.05%, Precision: 79.10%, Recall: 81.29%, F1: 80.18%, ROC-AUC: 98.49%	Consistent: State exact metrics confirmed by both documents.
Confusion Matrix	16,828 approved, 704 scams caught, 186 false alarms	TN: 16,828, FP: 186, FN: 162, TP: 704 (5-fold out-of-fold)	Consistent: Cite 16,828 TN, 704 TP, 186 FP, 162 FN.
Web Scraping Tool	"SerpAPI & Google Custom Search"	"BeautifulSoup DOM scraper + Brave Search / Google CSE"	Speak: "BeautifulSoup for website scraping, Brave Search/Google CSE for social auditing."
Chrome Extension	Manifest V3 Chrome Extension	Manifest V3 Chrome Extension (packaged in fake-job-detector-extension/)	Consistent: Chrome Extension built on Manifest V3.
Cloud Hosting	Render Platform	Render Platform (https://fake-job-detection-iuxn.onrender.com/#analyzer)	Consistent: Render Cloud Platform.

PART 8 — FINAL REVIEW CHEAT SHEET (READ BEFORE ENTERING)

Project in One Sentence: ShieldJob AI is a multi-modal fake job detection platform combining Balanced Random Forest ML (30%), Google Gemini 2.5 Flash (30%), BeautifulSoup website scraping (20%), and live social media auditing (20%) to deliver 0–100 safety scores via a Flask web app and Chrome Extension.

Problem: Rapid rise of recruitment fraud, fee extortion, and company impersonation on LinkedIn/Naukri where single-layer checks fail.

Solution: A 4-pillar verification ecosystem that cross-checks job text, semantic red flags, domain legitimacy, and social presence.

Technologies: Python 3.11, Flask, Scikit-Learn, Gemini 2.5 Flash, BeautifulSoup, HTML5, Vanilla CSS3, JS ES6, Chrome MV3, Render.

ML / NLP Specs: 17,880 EMSCAD samples, TF-IDF (12K features, unigrams+bigrams, sublinear_tf), Balanced Random Forest (100 trees, threshold 0.24).

Key Results: 98.05% Accuracy, 79.10% Precision, 81.29% Recall (704 scams caught), 80.18% F1, 98.49% ROC-AUC.

Challenges & Solutions: Class imbalance (4.84% scams) solved via balanced class weighting and threshold calibration (0.24); API latency solved via parallel requests and fallbacks.

My Contribution: End-to-end dataset preprocessing, ML model training & threshold tuning, Gemini API integration, website scraper, Flask web app, Manifest V3 Chrome Extension, and Render cloud deployment.

Limitations: Requires internet for API calls; external search API quotas; cannot inspect private Telegram chats.

Future Scope: Automatic DOM scraping without copy-paste; multilingual NLP; MCA registry verification; WhatsApp bot.

Top 10 Questions You Must Know By Heart

- What is the scoring formula?** $\text{Safety Score} = 0.30 \times \text{ML} + 0.30 \times \text{AI} + 0.20 \times \text{Website} + 0.20 \times \text{Social}$. Badges: ≥ 70 Safe, 50–69 Suspicious, < 50 Fake.
- How did you solve class imbalance?** Used `class_weight='balanced'` in Random Forest (giving $\sim 19.6\times$ weight to scam class) and calibrated threshold to 0.24.
- Why use both ML and Gemini AI?** ML gives fast statistical pattern matching on n-grams; Gemini gives contextual reasoning to catch covert fee demands and subtle fraud.
- What are your verified results?** 98.05% Accuracy, 79.10% Precision, 81.29% Recall, 80.18% F1-Score, 98.49% ROC-AUC on 17,880 samples.
- What dataset did you use?** EMSCAD (17,880 job postings: 17,014 real vs 866 fake) with 5-fold stratified cross-validation.
- Why did you choose TF-IDF with (1,2) n-grams?** Unigrams capture single words; bigrams capture critical phrases like 'wire transfer' and 'training fee'. Sublinear TF prevents high-frequency dominance.
- Where did you intern and for how long?** VedXience Innovations Pvt. Ltd., Hyderabad | 2 Months (May 6, 2026 – July 5, 2026) | Domain: Machine Learning with Python.
- How does the Chrome Extension work?** Built on Manifest V3; sends job text via HTTPS to Flask REST API `/api/extension/analyze` and displays score & badges in under 2 seconds.
- Where is it deployed?** Backend on Render Cloud (<https://fake-job-detection-iuxn.onrender.com/#analyzer>) with Unicorn WSGI.
- What is the difference between Precision and Recall in your model?** Recall is scam catch rate (81.29% / 704 caught); Precision is alarm accuracy (79.10% of flags are real scams).